

# plan

## Plan: HelixQA Full Integration into HelixCode System

### Overview

Deep, bluff-proof integration of the HelixQA submodule (with all its dependencies) into the HelixCode enterprise CLI agent system, with an example integration into the Catalogizer project. The goal is enterprise-grade QA coverage (100% of all supported test types and Challenges) that actually guarantees real functionality—not just passing tests that don't verify real usability.

---

### Stage 1 — Deep Repository Analysis (Parallel Research Swarm)

**Skill:** deep-research-swarm **Objective:** Exhaustive analysis of all three repositories and all documentation.

#### Sub-agents (all parallel):

1. **HelixCode\_Architecture\_Analyst:** Deep-dive into HelixCode repo structure, submodules, CLI architecture, client app types (Web, Desktop, Mobile, CLI, TUI), APIs/Services, configuration, build system, existing test infrastructure, and all MD docs.
2. **HelixQA\_Architecture\_Analyst:** Deep-dive into HelixQA repo—its testing framework, supported test types, Challenge system, screenshot capabilities, AI-driven QA sessions, dependencies, and all documentation.
3. **Catalogizer\_Integration\_Analyst:** Deep-dive into Catalogizer repo to understand how it integrates with HelixCode and how HelixQA can be added as a submodule.
4. **\*\*Docs\_&\_Guides\_Analyst\*\*:** Read ALL README, CLAUDE.MD, AGENTS.MD, manuals, user guides, architecture diagrams in all three repos.
5. **\*\*QA\_Types\_&\_Challenges\_Analyst\*\*:** Map every test type, Challenge type, and AI-driven QA capability in HelixQA; cross-reference with HelixCode's client matrix.
6. **\*\*UX\_&\_Enterprise\_Analyst\*\*:** Map the translation-tool UX, provider/model support, enterprise-grade requirements, and how QA must cover end-user workflows.

**Output:** Six detailed research briefs covering exact file paths, line ranges, architectural decisions, gaps, and integration points.

---

### Stage 2 — Cross-Validation & Gap Analysis

**Skill:** deep-research-swarm (synthesis phase) **Objective:** Cross-check findings, identify anti-bluff gaps, missing test coverage, and screenshot requirements.

## Sub-agents:

1. **Cross\_Validator**: Reconcile all six briefs, flag contradictions or omissions.
2. **Gap\_Analyst**: Identify every place where tests exist but don't actually verify real usability; map missing coverage for Web/Desktop/Mobile/CLI/TUI.
3. **Screenshot\_Requirements\_Analyst**: Define exact on-demand screenshot pipeline requirements per client app type during HelixQA sessions.
4. **\*\*Constitution\_ & Governance\_Analyst\*\***: Check all CLAUDE.MD, AGENTS.MD, Constitution files for anti-bluff testing mandates; identify missing ones.

**Output:** Validated, gap-free consolidated brief with risk register.

---

## Stage 3 — Master Integration Plan Writing

**Skill:** report-writing **Objective:** Produce the final, in-depth, phase-by-phase integration plan.

### Structure of the Plan:

- **Phase 0: Constitution & Governance Update** — Update CLAUDE.MD, AGENTS.MD, Constitution across all submodules with anti-bluff mandate.
- **Phase 1: Submodule Dependency Resolution** — Exact git submodule commands, dependency mapping, version locking.
- **Phase 2: HelixQA Core Integration into HelixCode** — Exact files, line references, configuration injection, CLI command registration.
- **Phase 3: Screenshot Pipeline Implementation** — On-demand screenshot architecture per client app (Web, Desktop, Mobile, CLI, TUI).
- **Phase 4: Test Type & Challenge Coverage Matrix** — 100% coverage mapping for every test type and Challenge.
- **Phase 5: Catalogizer Example Integration** — Step-by-step, line-by-line integration of HelixQA into Catalogizer.
- **Phase 6: Anti-Bluff Testing Framework** — Real-usability verification layer, synthetic-user workflows, end-to-end guarantees.
- **Phase 7: AI-Driven QA Session Orchestration** — How HelixQA drives heavy QA sessions across all clients and APIs.
- **Phase 8: Enterprise UX Validation** — Translation-tool UX validation across providers and models.
- **Phase 9: CI/CD & Automation Integration** — HelixQA in CI/CD, nightly heavy QA sessions, reporting.
- **Phase 10: Monitoring, Reporting & Compliance** — Dashboards, coverage reports, compliance with Constitution.

**Output:** helixqa-integration-plan.md (final master document).

---

## Stage 4 — Conversion to Professional Document

**Skill:** docx **Objective:** Convert the final Markdown plan to a professionally formatted Word document.

**Output:** helixqa-integration-plan.docx

---

## File Propagation

- Stage 1 outputs → Stage 2 inputs (all six briefs)
  - Stage 2 output → Stage 3 input (consolidated brief)
  - Stage 3 output → Stage 4 input (helixqa-integration-plan.md)
- 

## Anti-Bluff Mandate

Every recommendation in the final plan MUST include: 1. Exact file path and line range 2. What the current code does 3. What the gap/risk is 4. Exact change to make 5. How the change guarantees real usability (not just test-passing) 6. Challenge/test that proves real usability

This mandate MUST be codified in all Constitution/CLAUDE.MD/AGENTS.MD files.